

# Computer Programming Summer Interest Group

---

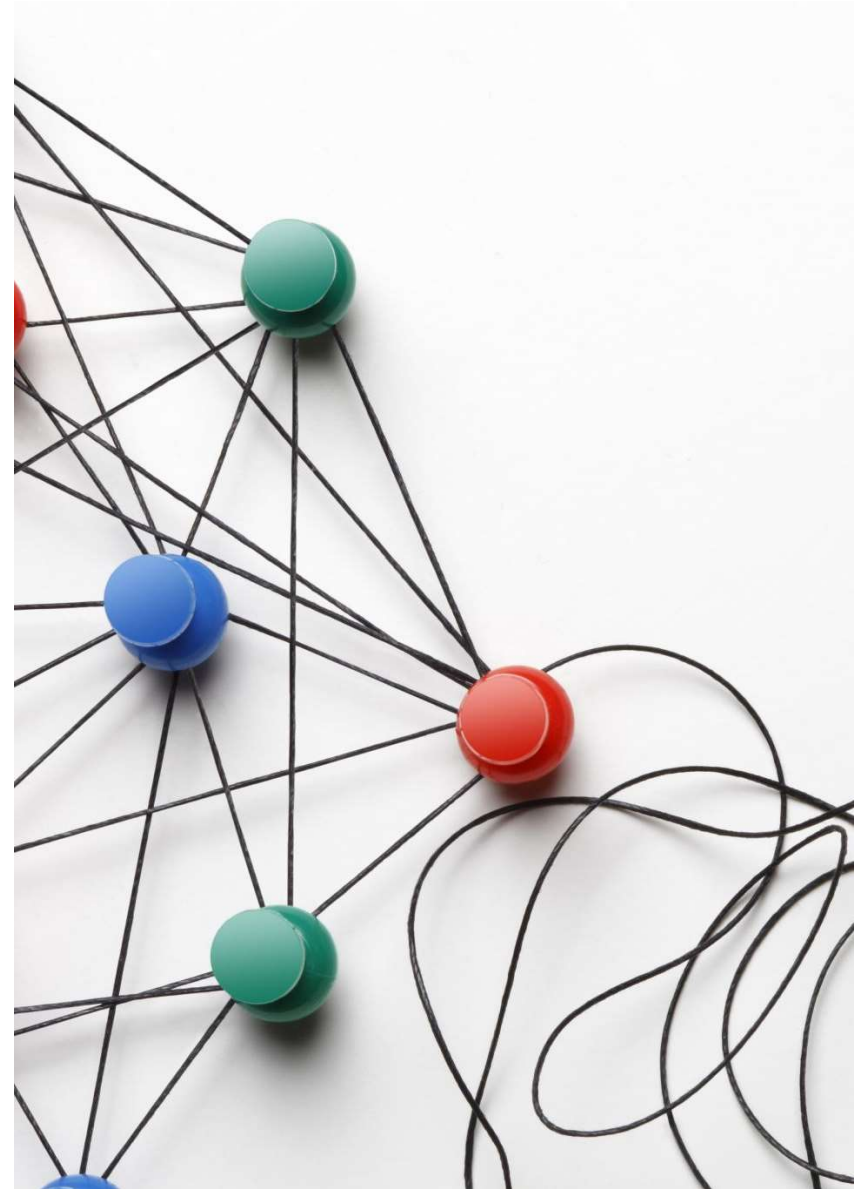
Session 12 / Comprehensions

## Matthew 22:34-40

But when the Pharisees heard that he had silenced the Sadducees, they gathered together. And one of them, a lawyer, asked him a question to test him. “Teacher, which is the great commandment in the Law?” And he said to him, “You shall love the Lord your God with all your heart and with all your soul and with all your mind. This is the great and first commandment. And a second is like it: You shall love your neighbor as yourself. On these two commandments depend all the Law and the Prophets.”

# Objectives

- By the end of this session, you will be able to:
  - Understand the syntax and structure of list comprehensions.
  - Learn how to use set comprehensions for unique collections.
  - Explore dictionary comprehensions for mapping keys to values.
  - Compare comprehensions with traditional for-loops.
  - Practice creating and using comprehensions in practical exercises.



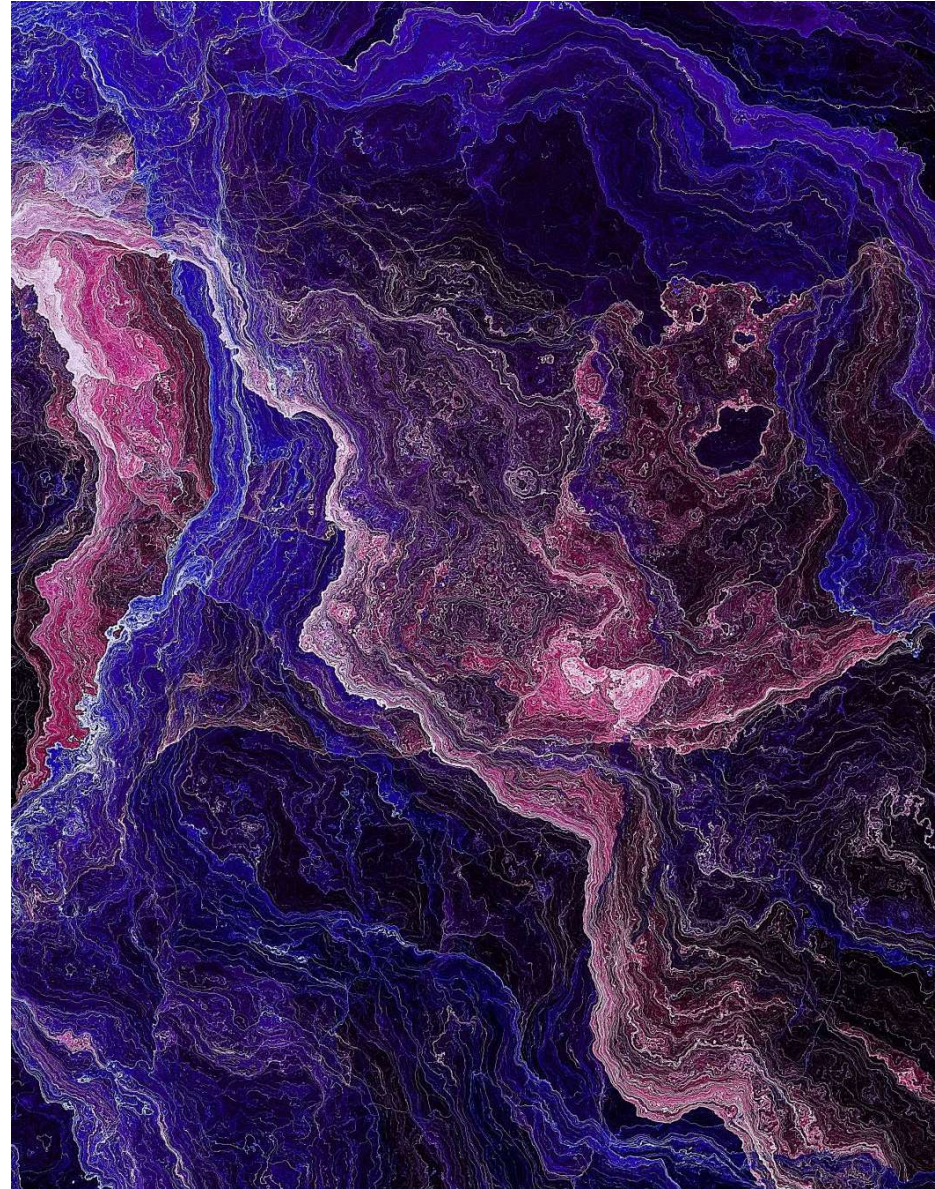
# Python Comprehensions

- Python comprehensions are a concise method to initialize standard collections
  - Lists
  - Tuples
  - Dictionaries
  - Sets
- This approach is called “comprehension”
- Additional reading: [List Comprehension](#)



# Background

- The notion of a Python comprehension is derived from mathematics
- In set notation, mathematicians define a set based on how the elements are defined:
  - $X = \{x^2 \mid x \in \mathbb{Z}^+, x < 10, x \text{ is odd}\}$
  - Means: X is the set of integers  $x^2$  such that  $0 < x < 10$  and  $x$  is odd.
- Python comprehensions borrow this conciseness to build python lists, sets, tuples and dictionaries



# List Comprehension

- Consider generating a list of the squares of odd integers  $n$ , where  $0 \leq n < 10$ .

## Traditional approach:

```
odd_squares = []
for n in range(10):
    if n % 2 == 1:
        odd_squares.append(n * n)

print(odd_squares)
```

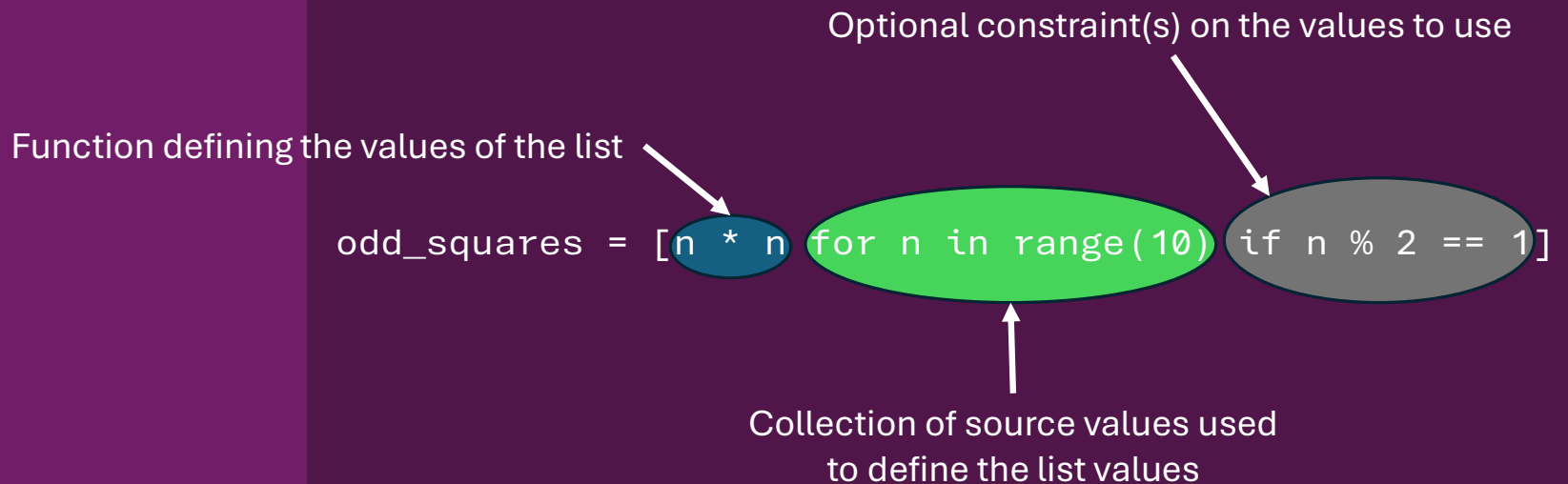
## Using list comprehension:

```
odd_squares = [n * n for n in range(10) if n % 2 == 1]

print(odd_squares)
```

## Syntax of a List Comprehension

- Consider generating a list of the squares of odd integers  $n$ , where  $0 \leq n < 10$ .



The entirety of the part inside the brackets is known as the “generator”

# Tuple comprehensions

- Python tuples are immutable
  - Once assigned a value, they cannot change
    - well, unless the tuple contains mutable objects... but let's not go there
- Using the traditional approach to define a list (successively adding elements) won't work on a tuple.
- Python provides no direct syntax for generating a tuple as a comprehension
- But we can place the generator inside a constructor for a tuple to achieve the same result

```
tuple(n**2 for n in range(10) if n % 2 == 1)
```



# Set Comprehension

## Traditional approach:

```
odd_squares = set()
for n in range(10):
    if n % 2 == 1:
        odd_squares.add(n * n)
```

## Using set comprehension:

```
odd_squares = {
    n * n
    for n in range(10)
    if n % 2 == 1}
```

**Note the use of braces, rather than brackets to designate a set, rather than a list**

## Dictionary Comprehension

- Dictionary comprehension can be used to create dictionaries.
- Use the braces (like in the case of sets)
- Provide both the key and value separated by a colon (i.e. ':')

```
odd_squares = {}  
for n in range(num):  
    if n % 2 == 1:  
        odd_squares[n] = n**2
```

```
odd_squares = {  
    n: n**2  
    for n in range(num)  
    if n % 2 == 1}
```



# Activities

- Write a program which analyzes the the text from “philemon\_esv.txt” from the github page
  - Read in the text of the file
  - Generate a dictionary using comprehension to count the number of occurrences of each word in the document
- Hints:
  - The text should contain only upper-case letters and spaces (no line breaks or other special characters or numbers)
  - The list method “count” may be useful
  - Recall sets contain only unique values